Stellenbosch
UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

Backseat driver: Privledged-information auxiliary loss in autonomous racing

by

Jacques Friedrich Amsel

Report submitted in partial fulfilment of the requirements of the module Project (E) 448 for the degree Baccalaureus in Engineering in the Department of Electrical and Electronic Engineering at the University of Stellenbosch

Study leader: Prof. JC Schoeman

October 2026

Acknowledgements

Plagarism declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektueleeiendom van ander persone asof dit jou eie werk is. *Plagiarism is the use of ideas, material and other intellectual property of another's work and to present is as my own.*
2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is. *I agree that plagiarism is a punishable offence because it constitutes theft.*
3. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelikse aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is. *Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.*
4. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie. *I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.*

26938839 Student number	 Signature
J.F. Amsel Initials and surname	16 October 2026 Date

Abstract

Table of contents

Acknowledgements i

Plagarism declaration ii

Abstract iii

Notation vi

Acronyms and abbreviations viii

1. Introduction 1

1.1. Background and motivation 1

1.2. Problem statement & objectives 2

2. Background and Related Literature 3

2.1. Partially Observable Markov Decision Processes (POMDPs) . 3

2.1.1. Value functions 4

2.2. Reinforcement Learning (RL) 5

2.3. Policy gradient methods 6

2.3.1. REINFORCE 7

2.3.2. Proximal Policy Optimization (PPO) 8

2.4. Auxiliary Learning in Policy-Based Methods 9

2.4.1. Auxiliary Losses 9

2.4.2. Auxiliary Rewards/Shaped Rewards 10

2.5. Loss scaling 11

2.5.1. Hyperparameter tuning 11

2.5.2. Uncertainty weighting 11

2.6. Huber loss 13

2.7. Linear Probing of Representations 13

2.8. Previous work 14

3. System overview 15

3.1. Simulator 16

3.1.1. Simulator performance comparison 16
3.1.2. Unity Vehicle Model 17
3.1.2.1. The WheelCollider 17
3.1.2.2. RigidBody 19
3.1.3. Track generation 19
3.2. Training performance tuning 21

4. Auxiliary heads **23**
4.1. The model 23

5. Experiments & results **25**
5.1. Probing intermediate layers for auxiliary information 25
5.2. Meeting questions 27
5.3. Learned features after auxiliary learning 28
5.4. Primary Task Performance 29
5.4.1. Variation in Primary Task Performance 30
5.5. Normalisation of ego-agent privledged information 30

6. What I did **31**

7. Conclusions and recommendations **32**

Bibliography **33**

Appendix **39**
Appendix A WheelCollider properties 39
Appendix B Auxiliary Head Regression Results 40
Appendix C Training Performance Benchmark Results 41

Notation

γ	Discount rate
λ	GAE discount rate
r_t	Reward received at time t
$G_t = \sum_{i=t}^{T} \gamma^{i-t} \cdot r_i$	Discounted return at time t
$\mathcal{S}$	Set of all states
S_t	Random variable for state at time t
s_t	State at time t
$\mathcal{O}$	Observation space
O_t	Random variable for agent's observation at time t
o_t	Agent's observation at time t
$\mathcal{A}$	Action space
A_t	Random variable for action taken at time t
a_t	Observed action taken at time t
π_θ	Policy given parameters θ
$\pi_\theta(a_t \mid o_t)$	Probability density for action a_t given observation o_t and policy parameters θ
$\Delta_{\mathcal{Y}}(X)$	Set of all probability distributions over variable X with support $\mathcal{Y}$
$V^\pi(s) = \mathbb{E}_\pi[R_t \mid S_t = s]$	Value function for policy π at state s
$Q^\pi(s, a) = \mathbb{E}_\pi[R_t \mid S_t = s, A_t = a]$	Action-value function for policy π at state s and action a
$\hat{X}$	Estimator for random variable X

$\hat{x}$	Sample estimate of random variable X
$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$	Advantage function for policy π at state s and action a
$\mathrm{Var}(X)$	Variance of random variable X
$\rho_0(s)$	Initial state distribution
$X \sim Y$	Random variable X is distributed according to distribution Y
$X \dot{\sim} Y$	Random variable X is approximately distributed according to distribution Y
$\mathbb{R}$	The set of real numbers
$\mathbb{E}[X]$	Expected value of random variable X
$\mathbb{E}_{x\sim p}[f(x)]$	Expected value of function $f(x)$ when x is distributed according to p
$N(y;\mu;\sigma^2)$	Gaussian probability density function with mean μ and variance σ^2 with evaluation point y

Acronyms and abbreviations

LiDAR	Light Detection and Ranging
MDP	Markov Decision Process
MAE	Mean Absolute Error
MSE	Mean Squared Error
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
RMSE	Root Mean Squared Error

Chapter 1

Introduction

1.1. Background and motivation

Autonomous racing is a challenging problem in autonmous robotics: the high speed, tight control margins, and short time available to recover from bad decisions makes for a demanding testbed to develop and evaluate autonomous driving algorithms. Studying autonomous racing is a proxy for development of autonomous driving algorithms, which have the potential to reduce severe and fatal accidents (e.g. severe lane-departure accidents [1]), improve traffic flow [2], and reduce emissions [3].

Generally, there exist two broad approaches to autonomous driving: end-to-end learning, where a model is trained to map sensor inputs directly to control outputs, and modular pipelines, where the problem is broken down into subproblems such as perception, planning, and control. End-to-end learning has the advantage of being able to learn a policy directly from data, without requiring hand-engineered features or intermediate representations. However, it can be difficult to interpret and debug, and may require large amounts of data to learn effectively. Modular pipelines, on the other hand, can be more interpretable and easier to debug, but may require more domain knowledge and engineering effort to design and tune [4].

Auxiliary learning is a technique involving training a model to perform additional (auxiliary) tasks alongside the primary task, in the hope that these tasks will help the model learn representations that are useful for the primary task. In some cases, it can be used to improve the performance of reinforcement learning models by providing additional supervisory signals during training that improve the model's learned representations [5], while also aiding in interpretability of the learned representations [4].

1.2. Problem statement & objectives

An end-to-end RL agent that drives from LiDAR alone has to learn a representation of privledged information it can't directly observe in the real world (e.g. its own velocity and slip, the curvature ahead, and its opponent's location in the case of head-to-head (H2H) racing). The agent may struggle to learn these representations early in training, when it has not yet learned to drive well enough to explore the environment effectively, which can lead to slow convergence.

However, since the privledged information is available in simulation, we can use auxiliary learning to provide additional supervisory signals to the agent during training, which may help it learn better representations of the privledged information and improve its performance or training speed.

The objectives of this Skripsie are to (a) select an appropriate simulation environment for autonomous racing, (b) implement a baseline end-to-end RL agent that drives from LiDAR alone, (c) gain an understanding of what representations the baseline agent natively encodes, (d) implement auxiliary learning to provide additional supervisory signals to the agent during training, and (e) evaluate the performance of the auxiliary learning agent against the baseline agent.

Chapter 2

Background and Related Literature

2.1. Partially Observable Markov Decision Processes (POMDPs)

Many control and robotics problems require an agent to make a sequence of decisions in an environment, where each decision influences what happens next and where the agent can sense only part of the environment. We model such problems as a *partially observable Markov decision process* (POMDP) defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \rho_0, \mathcal{R}, \gamma, \mathcal{Z})$ [6], whose interaction loop is shown in Figure 2.1.

At each time step t, the environment is in state $s_t \in \mathcal{S}$, where $\mathcal{S}$ is the state space. This state is a *complete* description of the environment at that time. The first state is sampled from an initial state distribution, $s_0 \sim \rho_0$. The agent does not see s_t directly. Instead, it receives an observation $o_t \in \mathcal{O}$ drawn from the observation function, $o_t \sim \mathcal{Z}(\cdot \mid s_t)$, where $\mathcal{O}$ is the observation space. The information in s_t that is missing from o_t is called *privileged information*. For example, a car may observe its lidar measurements, but not the road surface conditions directly.

Based on its observation, the agent chooses an action $a_t \in \mathcal{A}$, where $\mathcal{A}$ is the action space. It does so using a *policy* π, which maps an observation to a probability distribution over actions; we write $a_t \sim \pi(\cdot \mid o_t)$. The policy is what we want to find: it describes the agent's behaviour, whereas the POMDP describes the environment in which the agent acts. (For simplicity, we let the policy depend only on the current observation; should the policy depend on a history of observations, we would write $\pi(\cdot \mid o_t, o_{t-1}, ...)$.) After the agent has taken action a_t, the environment moves to a new state, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$, where P is the transition function. By the *Markov property*, the next state depends only on the current state and action, not on earlier history [7]. Then, the agent receives

a scalar reward $r_t = R(s_t, a_t, s_{t+1})$, where R is the reward function, together with the next observation o_{t+1}. The agent's goal is to maximise the expected discounted return $\mathbb{E}[G_0] = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$, where $\gamma \in [0, 1)$ is a discount factor. The return represents a weighted sum of all future rewards, where later rewards are worth less than immediate ones. The discount rate γ describes how much the agent values future rewards relative to immediate ones. The policy that maximises the expected discounted return is called the *optimal policy* π^*.

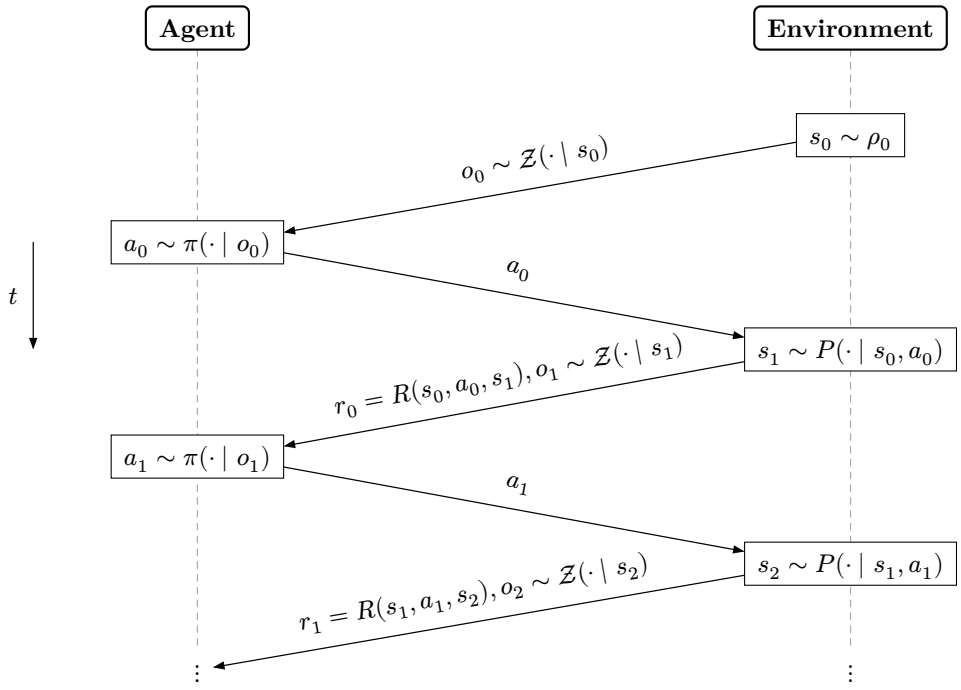


Figure 2.1: Interaction between the agent and the environment over time (t).

2.1.1. Value functions

Throughout this section we assume a memoryless policy $\pi(\cdot \mid o_t)$. Since $o_t = \mathcal{Z}(s_t)$, the agent's action distribution in state s is $\pi(\cdot \mid \mathcal{Z}(s))$, and because the state is Markov, the expected return from time t depends only on s_t. The value functions below are therefore well defined as functions of the state.

Value functions are a central concept in reinforcement learning, providing a measure of how good it is for the agent to be in a particular state or to take a particular action. The state-value function,

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

represents the expected discounted return, the agent can achieve starting from state s and following policy π thereafter. The action-value function,

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

represents the expected discounted return the agent can achieve starting from state s, taking action a, and following policy π thereafter. The advantage function,

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s) \tag{1}$$

measures the relative value of taking action a in state s compared to the average value of all actions in that state .

These state-value and action-value functions satisfy the Bellman equations, which express the relationship between the value of a state and the values of its successor states. [8] The Bellman equation for the state-value function is given by:

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s] \tag{2}$$

and the Bellman equation for the action-value function is given by:

$$Q^\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \tag{3}$$

These Bellman equations Equation 2 and Equation 3 provide a recursive relationship that allows the agent to update its value estimates based on the observed rewards and the estimated values of successor states.

2.2. Reinforcement Learning (RL)

Reinforcement learning (RL) is a subfield of machine learning concerned with finding a policy π that maximises the expected discounted return $J(\pi) = \mathbb{E}_\pi[G_0]$ using only data sampled from interaction with the environment, without labelled optimal actions or a known model. RL differs from supervised learning in that feedback is evaluative rather than instructive: the agent is told how good its actions were, not which action was correct, and rewards may arrive long after the actions that caused them. Furthermore, the data distribution depends on the agent's own policy, so the agent must explore.

RL algorithms fall into three broad classes. *Value-based* methods learn the optimal action-value function $Q^*(s,a) = \max_\pi Q^\pi(s,a)$, which satisfies the Bellman optimality equation

$$Q^*(s,a) = \mathbb{E}\Big[r_t + \gamma \max_{a' \in \mathcal{A}} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a\Big],$$

and act greedily with respect to it, $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^*(s,a)$. In a fully observable MDP this greedy policy is optimal, but the agent is restricted to deterministic policies, and the maximisation over $\mathcal{A}$ can be intractable for continuous actions [9]. In a POMDP, the observation is not necessairily Markov, so $Q(o,a)$ does not satisfy a Bellman equation, and a state-based $Q(s,a)$ cannot define a deployable policy as the agent

does not observe s. Value-based methods therefore require a Markov input such as the observation history. *Policy-based* methods instead learn a parameterised policy π_θ directly by ascending the gradient of $J(\pi_\theta)$ with respect to θ, and do not need the Markov property of the observations or an $\arg\max$ over actions. They can also represent stochastic policies, which may be necessary for optimal behaviour in some POMDPs [10] (although there always exists a deterministic optimal policy in fully-observed MDPs). *Actor-critic* methods combine both: a learned value function (the critic) is used to reduce the variance of the policy gradient, and since the critic is only used in training, it may condition on privileged information. We focus on policy-based and actor-critic methods in this report [7].

2.3. Policy gradient methods

Policy gradient methods are a class of RL training algorithms that learn a policy π_θ with parameters θ directly by maximising the expected return $\mathbb{E}[G_0]$ with respect to the policy parameters θ. Given a trajectory $\tau = (s_0, o_0, a_0, r_0, s_1, o_1, a_1, r_1, ..., s_{T-1}, o_{T-1}, a_{T-1}, r_{T-1}) \sim p_\theta$, where p_θ is the probability of the trajectory when following the policy π_θ, we intend to maximise the expected return with respect to the policy parameters θ:

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[G_0] = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T}\gamma^t r_t\right] = \int p_\theta(\tau)G(\tau)\,\mathrm{d}\tau$$

Using the *log-derivative* trick, $\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)$, and assuming that the integral and derivative can be interchanged[1], the gradient of the expected return with respect to θ is then

$$\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau)G(\tau)\,\mathrm{d}\tau = \int \nabla_\theta p_\theta(\tau)G(\tau)\,\mathrm{d}\tau = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)G(\tau)\,\mathrm{d}\tau$$
$$= \mathbb{E}_{\tau\sim p_\theta}[\nabla_\theta \log p_\theta(\tau)G(\tau)]$$

Considering that the probability of a trajectory can be factorised as

$$p_\theta(\tau) = p(s_0)\prod_{t=0}^{T-1}\underbrace{p(o_t \mid s_t)}_{\mathcal{Z}}\cdot\pi_\theta(a_t \mid o_t)\cdot\underbrace{p(r_t \mid s_t, a_t, s_{t+1}, o_t)}_{\mathcal{R}}\cdot\underbrace{p(s_{t+1} \mid s_t, a_t)}_{\mathcal{P}}$$

since $p(s_0)$, $\mathcal{P}$, $\mathcal{Z}$ and $\mathcal{R}$ do not depend on θ, their gradients vanish, so

[1] This can be done only under certain regularity conditions not explored in this report, but generally if the functions are continuous and differentiable everywhere, and the integral is finite.

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid o_t)$$

Then, the *policy gradient* can be expressed as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid o_t) G(\tau) \right]$$

Write $G(\tau) = G_0$. Splitting the return at time t,

$$G_0 = \sum_{k=0}^{t-1} \gamma^k r_k + \gamma^t G_t, \quad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k,$$

where G_t is the discounted *reward-to-go*,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid o_t) \left(\sum_{k=0}^{t-1} \gamma^k r_k + \gamma^t G_t \right) \right]$$

However, since the expectation of the fixed past-reward terms is zero, they can be removed from the policy gradient without introducing bias. This can be seen by conditioning on the history $h_t = (s_0, o_0, a_0, r_0, ..., s_{t-1}, o_{t-1}, a_{t-1}, r_{t-1}, s_t, o_t)$:

$$\mathbb{E}_{a_t \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid o_t) \mid h_t] = \int_a \nabla_\theta \pi_\theta(a \mid o_t) = \nabla_\theta \int_a \pi_\theta(a \mid o_t) = \nabla_\theta 1 = 0,$$

The past-reward terms therefore vanish in expectation, leaving

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid o_t) G_t \right].$$

Removing these zero-mean terms typically reduces the variance of the Monte Carlo estimate. In practice the factor γ^t is usually dropped; this yields a biased estimate of $\nabla_\theta J(\theta)$ for $\gamma < 1$ [11].

2.3.1. REINFORCE

The REINFORCE algorithm for policy learning was introduced by Williams in 1992 in the journal *Machine Learning* [12]. It is one of the simplest policy gradient methods: by sampling N trajectories $\tau^{(i)} \sim \pi_\theta$ of length T_i timesteps each, the policy gradient can be estimated in a Monte-Carlo fashion:

$$\widehat{\nabla_\theta J(\theta)} = \frac{1}{\sum_{i=0}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \log\Big(\pi_\theta\Big(a_t^{(i)} \mid o_t^{(i)}\Big)\Big) \cdot G_t^{(i)}$$

where $\overline{T}$ is the average trajectory length. In order to run REINFORCE in an AutoGrad engine (such as PyTorch or TensorFlow), a loss function $\mathcal{L}_{\text{REINFORCE}}$ is defined such

that its gradient with respect to the policy parameters θ is equal to the policy gradient estimate above. This loss function is given by:

$$\mathcal{L}_{\text{REINFORCE}} = -\frac{1}{\sum_{i=0}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \log\Big(\pi_\theta\Big(a_t^{(i)} \mid o_t^{(i)}\Big)\Big) \cdot G_t^{(i)}$$

Although REINFORCE is simple to implement, it has several drawbacks. First, it has high variance in its gradient estimates, which can lead to slow convergence and instability during training. Second, it does not make use of a value function to estimate the expected return, which can further increase variance. Finally, it is an on-policy algorithm, meaning that it can only learn from trajectories generated by the current policy, which can limit exploration and learning efficiency (since it cannot train multiple epochs on the same rollout).

2.3.2. Proximal Policy Optimization (PPO)

PPO is a modern algorithm for policy learning introduced in 2017 by researchers at OpenAI [13]. It improves on REINFORCE in two ways. First, it is an actor-critic method: alongside the policy, it learns a value function $V^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate. Second, it optimises a *clipped surrogate objective*, $\mathcal{L}_{\text{PPO}}$, that limits how far each update can move the policy. This prevents large, destructive steps and makes several epochs of updates on the same rollout possible.

Recall that the advantage function in Equation 1 is the difference between the action-value and state-value functions. It therefore measures how much better (positive) or worse (negative) an action is than the policy's average action in that state. In training, $\hat{A}_t(s_t, a_t)$ is estimated from observed rewards and the learned value function, $\hat{V}^\pi(s)$, for example with Generalized Advantage Estimation [14]. In PPO, the clipped surrogate objective is given by

$$\mathcal{L}_{\text{PPO}} = -\frac{1}{\sum_{i=0}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \min\Big(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t\Big)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}$ is the ratio of the new policy's probability of taking action a_t given observation o_t to the old policy's, and ε is a hyperparameter that controls how much the policy is allowed to change in a single update. The clip function ensures that a policy update that would make $r_t(\theta)$ fall outside of the interval $[1-\varepsilon, 1+\varepsilon]$ is clipped to the nearest endpoint of that interval. This prevents large policy updates that could

destabilise training. The min function on the other hand ensures that, while the policy's update is capped when an update makes a good action ($A_t > 0$) more likely, it is not protected from penalties if an update makes a bad action ($A_t < 0$) more likely.

2.4. Auxiliary Learning in Policy-Based Methods

In RL, the *primary task* is to update a policy π_θ to maximises the expected discounted return $\mathbb{E}_{\tau\sim\pi_\theta}[G_0]$. *Auxiliary learning* augments this with *auxiliary tasks* whose learning signals are used only to improve the agent's representations. Unline general multi-task learning, perfromance on the auxiliary tasks is not itself the goal, and their outputs are typically discarded at inference time. Because auxiliary tasks can supply training signal when the reward is sparse or delayed [15], they can improve sample efficiency, and, by acting as an inductive bias[2], generalisation [17]. We consider two formulations: auxiliary **losses**, which leave the RL objective unchanged and shape only the shared representation, and auxiliary **rewards**, which modify the objective itself and can therefore change the optimal policy.

2.4.1. Auxiliary Losses

In this formulation, the agent is trained on a weighted sum of the primary loss and K auxiliary losses. Each loss is computed by a separate *auxiliary head* that reads from a representation shared with the policy. Writing the parameters as $\theta = \left(\theta_{\mathrm{s}}, \theta_\pi, \theta_{\mathrm{a}}^{(1)}, ..., \theta_{\mathrm{a}}^{(K)}\right)$, where θ_{s} are the shared (encoder) parameters, θ_π the policy head and $\theta_{\mathrm{a}}^{(k)}$ the k-th auxiliary head, the total loss is

$$\mathcal{L}_{\mathrm{total}}(\theta) = \mathcal{L}_{\mathrm{primary}}(\theta_{\mathrm{s}}, \theta_\pi) + \sum_{k=1}^{K} w^{(k)} \mathcal{L}_{\mathrm{aux}}^{(k)}\left(\theta_{\mathrm{s}}, \theta_{\mathrm{a}}^{(k)}\right),$$

where $w^{(k)} \geq 0$ weights each auxiliary task relative to the primary loss (only relative weights matter, so the primary weight is fixed to 1). The gradients with respect to each parameter group are

$$\nabla_{\theta_\pi} \mathcal{L}_{\mathrm{total}} = \nabla_{\theta_\pi} \mathcal{L}_{\mathrm{primary}},$$

$$\nabla_{\theta_{\mathrm{a}}^{(k)}} \mathcal{L}_{\mathrm{total}} = w^{(k)} \nabla_{\theta_{\mathrm{a}}^{(k)}} \mathcal{L}_{\mathrm{aux}}^{(k)}, \quad \nabla_{\theta_{\mathrm{s}}} \mathcal{L}_{\mathrm{total}} = \nabla_{\theta_{\mathrm{s}}} \mathcal{L}_{\mathrm{primary}} + \sum_{k=1}^{K} w^{(k)} \nabla_{\theta_{\mathrm{s}}} \mathcal{L}_{\mathrm{aux}}^{(k)}.$$

[2]Inductive bias is the set of assumptions a learning algorithm makes to predict outputs given inputs that it has not encountered before. [16]

Only the shared parameters θ_s receive gradients from both the primary and auxiliary losses, which is how the auxiliary tasks influence the policy: they shape the representation it is built on

The primary loss, such as $\mathcal{L}_{\text{PPO}}$, is still a policy-gradient surrogate with a high-variance gradient estimate. Auxiliary losses, by contrast, are usually ordinary supervised or self-supervised objectives (for example predicting the next observation, or the immediate reward), so their gradients are exact and low-variance, and do not depend on the reward being observed [citation]. This is what makes them useful when rewards are sparse or delayed. Their cost is that the auxiliary and primary gradients can conflict on the shared parameters, θ_s so the weights $w^{(k)}$ must be chosen with care.

2.4.2. Auxiliary Rewards/Shaped Rewards

Alternatively, the auxiliary task can be incorporated into the reward. The agent is given the modified reward

$$r_t^{\text{total}} = r_t^{\text{primary}} + \sum_{k=1}^{K} w^{(k)} r_t^{\text{aux},(k)},$$

where each $r_t^{\text{aux},(k)}$ rewards behaviour that is expected to help with the primary task, such as an intrinsic curiosity bonus [18] or progress towards a subgoal. The policy is then trained to maximise the expected return of r_t^{total} with the same policy gradient machinery as before.

Unlike auxiliary losses, this changes the objective being optimised: the agent now solves a different (PO)MDP, whose optimal policy need not be the same as π^* for the primary reward. Poorly chosen auxiliary rewards can therefore lead the agent to exploit them at the expense of the primary task (*reward hacking*). The exception is *potential-based shaping*, where the auxiliary reward has the form $\gamma\Phi(s_{t+1}) - \Phi(s_t)$ for some potential function Φ, which is guaranteed to leave the optimal policy unchanged [19].

The auxiliary rewards do influence the policy, but only indirectly through the return G_t, and therefore through the same high-variance score-function estimator as the primary reward. No exact gradient is available, and the agent must work out through its own exploration which auxiliary rewards actually contribute to the primary task. This credit-assignment problem is harder still when the auxiliary rewards are themselves sparse or delayed.

2.5. Loss scaling

When composing the L_{total}, the question naturally arises: what should be the values of the weights $\{w^{(1)}, ..., w^{(N)}\}$? Since the values cannot simply be learnt by gradient descent (they would simply shrink to 0 or go negative to lower $\mathcal{L}_{\text{total}}$), we must use another technique.

2.5.1. Hyperparameter tuning

One may empirically estimate the optimal weights using hyperparameter tuning tools such as Optuna [20], or a simple grid search. However, when it is expensive to train a model, this method of search quickly becomes impractical, since every candidate set of weights requires its own training run.

Hyperparameter tuners differ in how they choose which weights to try next. Grid search evaluates every combination on a fixed N-dimensional grid, so its cost grows exponentially with the number of weights N. Random search samples configurations at random and is often more efficient than grid search, because usually only a few hyperparameters matter much [21]. Bayesian optimisation methods go further by building a model of how the score depends on the hyperparameters and using it to choose promising candidates. Examples include Gaussian-process-based optimisation [22] and the tree-structured Parzen estimator (TPE) [23], which is the default sampler in Optuna. Methods such as Hyperband [24] and its asynchronous variant ASHA [25] give each trial a small budget and only continue the best-performing ones. However, this method may kill RL trials that perform more exploration early on, so are not ideal for such scenarios.

Even with these savings, each trial still involves training a model, and the number of trials needed grows with the number of loss terms N. For multi-task models that are expensive to train, this motivates methods that learn the weights during training instead.

2.5.2. Uncertainty weighting

[Will only include this if I have time to implement UW] Two kinds of uncertainty can be modelled in a model's predictions [26]:

- *Epistemic* uncertainty: the model is unsure because it hasn't been trained on enough data. More data reduces epistemic uncertainty.

- *Aleatoric* uncertainty: the data itself is noisy or incomplete, so even a perfect model can't predict exactly. In this case, more training data doesn't help.

Aleatoric uncertainty has two forms:

- *Data-dependent* or *Heteroscedastic*: the noise changes from input to input (e.g. a blurry image is harder to make inference from than a clear one).
- *Task-dependent* or *Homoscedastic*: the noise is the same for every input and depends only on the task

Kendall et al. propose a solution to determining the weights $\{w_1, ..., w_N\}$ problem by modelling the homoscedastic (task-dependent) uncertainty only [27].

Instead of formulating the prediction for task i, $\hat{y_i} = f_i(\boldsymbol{x})$, as a point estimate, it is modelled as a Gaussian with mean $f_i(x)$ and variance σ_i^2:

$$\hat{Y}_i \sim p(y_i \mid f_i(x)) = N(y_i; f_i(x); \sigma_i^2)$$

The negative log-likelihood of this Gaussian probability density function evaluated at the true value y_i is then the loss value for task i:

$$L_i = -\log p(y_i \mid f_i(x)) = \frac{1}{2\sigma_i^2} \|y_i - f_i(x)\|_2^2 + \log \sigma_i$$

A similar derivation can be made for L1 loss functions by assuming the predictions are $Y_i \sim \text{Laplace}(f_i(x), b_i)$:

$$L_i = -\log p(y_i \mid f_i(x)) = \frac{1}{b_i} \|y_i - f_i(x)\|_1 + \log b_i$$

Then, assuming the tasks are independent, the total loss is the sum of the individual losses. The log-variance $\log \sigma_i$ or log-scale parameter $\log b_i$ is a learnable parameter that is updated during training, allowing the model to learn the optimal weights for each loss term.

Of course, in reality, the tasks are not independent, but this method has empirically been shown to work well in practice [27]. With some caveats like the fact that it produces a bias towards easy tasks, since they result in smaller σ_i values. This has led to the development of other methods not explored in this Skripsie, such as so-called Optimal Uncertainty Weighting [28].

2.6. Huber loss

The Huber loss (with parameter β or in some literature k) is a loss function that is less sensitive to outliers compared to the standard Mean Squared Error (MSE) L2 loss [29]. It is defined as:

$$\mathcal{L}^{\beta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 \text{ if } |y - \hat{y}| \leq \beta \\ \beta \ |y - \hat{y}| - \frac{1}{2}\beta^2 \text{ otherwise} \end{cases}$$

In our case, since we have various loss terms, each relating to a different task where the scale of y differs, we must carefully choose the parameter β and scale each loss term appropriately.

Rewriting the loss to $\mathcal{L}^{\beta}\left(\frac{y-\mu}{\sigma}, \hat{y}\right)$ allows us to reduce the model's bias towards learning tasks with larger scales, since the loss is now computed on a normalised scale.

In actuality, PyTorch's implementation of the Huber loss (`smooth_l1_loss`) uses a rescaled definition, $\frac{\mathcal{L}^{\beta}(y,\hat{y})}{\beta}$, to prevent the β parameter from affecting the gradient in the linear region.

2.7. Linear Probing of Representations

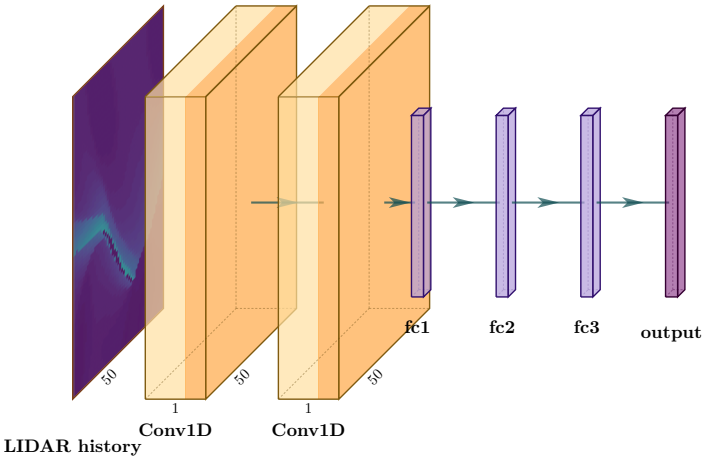


Figure 2.2: Example network architecture

Consider the network in Figure 2.2. Once trained, each layer l maps an input $\boldsymbol{x}$ to an intermediate representation $\boldsymbol{h}^{(l)} = f_l(\boldsymbol{x}; \theta) \in \mathbb{R}^{d_l}$. We would like to know what information these representations encode, and in particular whether they encode privileged state variables (e.g. velocity, slip, upcoming curvature) that.

Alain and Bengio [30] introduced *linear probes* for this purpose. The main network's weights are frozen, and a separate linear classifier (the probe) is trained on the representations $\boldsymbol{h}^{(l)}$ of each layer to predict the desired target. Since the probe cannot change

the representation, its test accuracy measures how much class information is *linearly* available at that layer. Anand et al. [31] extended this idea to reinforcement learning, probing an encoder's representations for ground-truth state variables of Atari games (read from the console's RAM) rather than for the network's own targets.

The choice of a *linear* probe is deliberate. A sufficiently expressive probe (e.g. a deep MLP) could compute the target from almost any input, including the raw observation, so high probe accuracy would say more about the probe than about the representation [32]. Restricting the probe to a linear map means that it can only succeed if the network has already done the work of encoding the target along some direction in the representation space.

Many of the privileged variables of interest are continuous rather than categorical, so we use a linear *regression* probe instead of a classifier, as done by McGrath et al. [33] to probe AlphaZero for chess concepts and by Gurnee and Tegmark [34] to probe language models for spatial and temporal encodings.

2.8. Previous work

Lample and Chaplot [35] explore a similar problem of training an RL agent to play doom using Deep Q-Networks (DQN) from first-person visual input. They find that auxiliary losses for presence of enemies in the agent's field of view significantly improves performance, approximately doubling agent's kill-to-death ratio. They, however, find that these auxiliary losses only significantly improve performance in the case of Deep *Recurrent* Q-Networks (DRQN), which receive single-frame inputs and are able to maintain a memory, and not in the case of DQNs, which are memoryless, and instead receive a stack of N previous observations as input. [Why do we not use DRQN?]

Chapter 3

System overview

The system trains an end-to-end RL agent to drive a simulated RoboRacer vehicle from LiDAR. It runs entirely in simulation and has two subsystems, shown in Figure 3.1: a Unity simulator, which models the vehicle and track, and a Python training script, which computes rewards and updates the policy. The two communicate over gRPC through the ML-Agents toolkit [36], [37].

The subsystems advance in lockstep, following the interaction loop in Figure 2.1. At each timestep, the simulator sends each agent's observation, along with privileged state information, to the training script. The script replies with the action each vehicle should take next. The privileged information is used only during training and is never given to the policy, so the trained agent can drive from LiDAR alone. Rewards are computed in Python rather than in Unity, so the reward function can be changed without rebuilding the simulator.

To speed up data collection, the simulator is replicated: each Unity instance contains `num_areas` independent training areas, and each trainer runs `num_inst` instances in parallel (the stacked boxes in Figure 3.1). The rest of this chapter describes the components in turn.

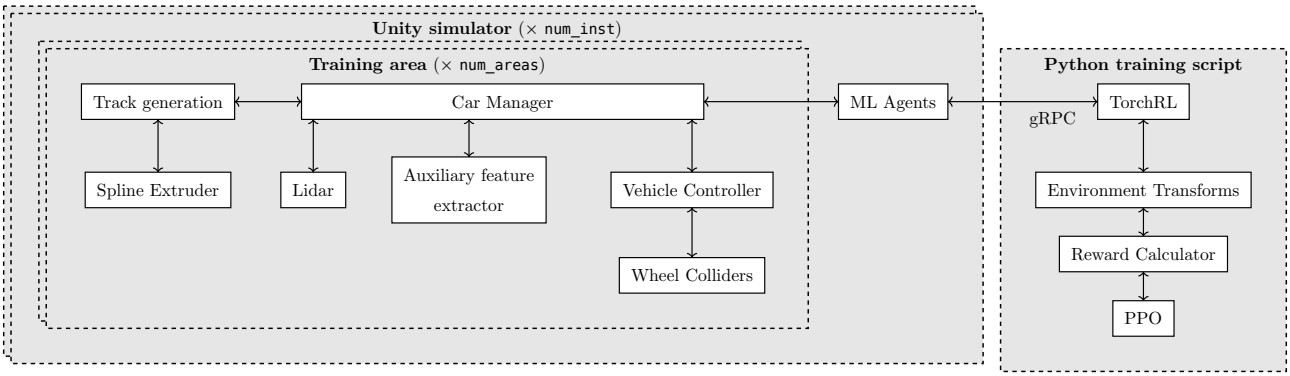


Figure 3.1: System diagram

3.1. Simulator

Previous work on end-to-end driving has largely made use of the F1Tenth gym [CITE], which is a Python-based 2D simulator for the F1Tenth (now RoboRacer) vehicle. This simulator is easy to use, and runs in Numba-compiled Python, but features a simplified bicycle model of the vehicle, which approximates slip dynamics with a linear function [38].

The AutoDrive simulator [39], which uses the Unity game engine, is a 3D simulator that models the vehicle and its environment using PhysX, a physics engine developed by NVIDIA. PhysX simulates rigid-body dynamics, including collisions, friction and gravity, and allows for higher-fidelity simulations, including weight transfer dynamics, and suspsension deformation.

	Unity (PhysX)	**F1Tenth gym**
Integrator	Runge-Kutta 4th order	Euler
Car model	4-wheel rigid body with suspension, weight transfer, non-linear wheel slip dynamics	2-wheel bicycle model with linearised slip dynamics
Sensors	Digital-Twin LiDAR, IMU, wheel encoders[3]	LiDAR, IMU (excl. pitch, roll)
Track generation	No	In Beta [40]

However, the AutoDrive simulator by default does not provide various features that are important for end-to-end training. For example, it is missing track generation & privledged information reporting, and its performance is slower than the F1Tenth gym. Therefore, custom simulator was implemented in Unity (albeit incorporating AutoDrive's `VehicleController` component, which provides an easy interface to modify parameters like mass, wheelbase, and all/rear/front-wheel drive toggles),.

3.1.1. Simulator performance comparison

In order to ensure the Unity simulator is suitable for training, and not just evaluation, its performance was compared to the F1Tenth gym. Both environments were configured for 100Hz physics and 25Hz control with one vehicle per environment. Then, a rollout of 10 000 control timesteps (400 sim-seconds) was performed in each environment on an M4

[3]The IMU and wheel encoders are modules available in the AutoDrive simulator, not native Unity

MacBook Air, and the average time per control step was measured. Both environments were run in headless mode.

Despite the Unity simulator being significantly more complex than the F1Tenth gym, it runs at a comparable speed, as shown in Figure 3.2. The Unity simulator is slower than the F1Tenth gym by 20%. The memory usage of the Unity simulator is also higher than that of the F1Tenth gym, but this is a marginal diffence of 115 MB on average.

Due to the relatively minor performance difference, and Unity's more realistic 3D physics, the Unity simulator was chosen for training the RL agent.

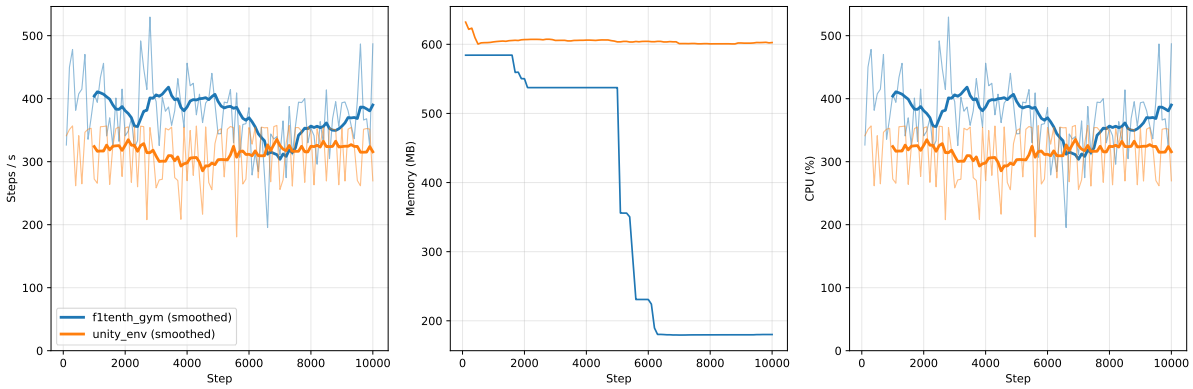


Figure 3.2: Benchmark of Unity simulator vs F1Tenth gym. Note that memory y-axis does not start at 0 to aid interpretability.

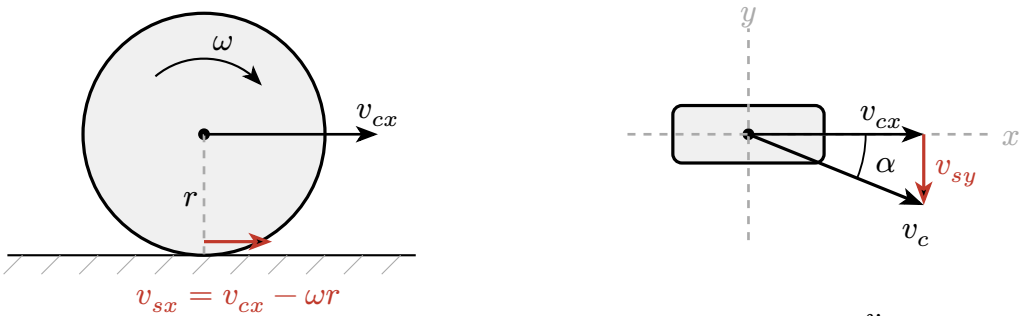
3.1.2. Unity Vehicle Model

The vehicle is modelled as a `RigidBody` with four wheels, each of which is simulated using a `WheelCollider`.

3.1.2.1. The WheelCollider

In our custom simulator, each wheel of the RoboRacer is modelled as a `WheelCollider`, which simulates the wheel's suspension, rotation and tyre friction [41]. This kind of collider uses physics similar to that modelled by the semi-empirical "magic formula" wheel dynamics proposed by Pacejka et al. in [42], and further described in [43].

The tyre model is built on the idea of *slip*. *Longitudinal slip*, κ, is the ratio between the longitudinal slip velocity and the wheel center velocity. *Lateral slip*, $\tan(\alpha)$, is the ratio between the lateral slip velocity and the wheel center velocity. A wheel slips when it spins faster than the wheel center moves (wheelspin), turns slower than the wheel center moves (braking), or is dragged sideways (cornering). A slip value of 0 therefore indicates a freely rolling wheel, while a slip value of > 1 indicates that the point of contact has a higher velocity than the wheel center.



(a) Side view: longitudinal slip **(b)** Top view: lateral slip

Figure 3.3: Slip quantities. Longitudinal slip compares the slip velocity at the contact point with the wheel-centre velocity; lateral slip compares the sideways slip velocity with the forward velocity.

A tyre needs some slip to produce force. The `WheelCollider` computes the force from the slip using a friction curve (Figure 3.4). Force rises with slip until it reaches a peak, the *extremum*, which is the tyre's maximum grip. Beyond the peak, the tyre begins to slide and the force falls to a lower, constant value, the *asymptote*. Pushing the car harder therefore gives less force, not more, which is a regime a racing agent must learn to avoid. The curve is defined separately for the forward direction (accelerating and braking) and the sideways direction (cornering). Note that in real life, at low slip values, the rubber stretches instead of actually sliding on the road. This is modelled by the rising part of the curve at low slip values.

Figure 3.4 shows the difference between a Unity-style piecewise friction curve and a Pacejka "magic formula" curve that was fit using least-squares to approximate Unity's friction curve.

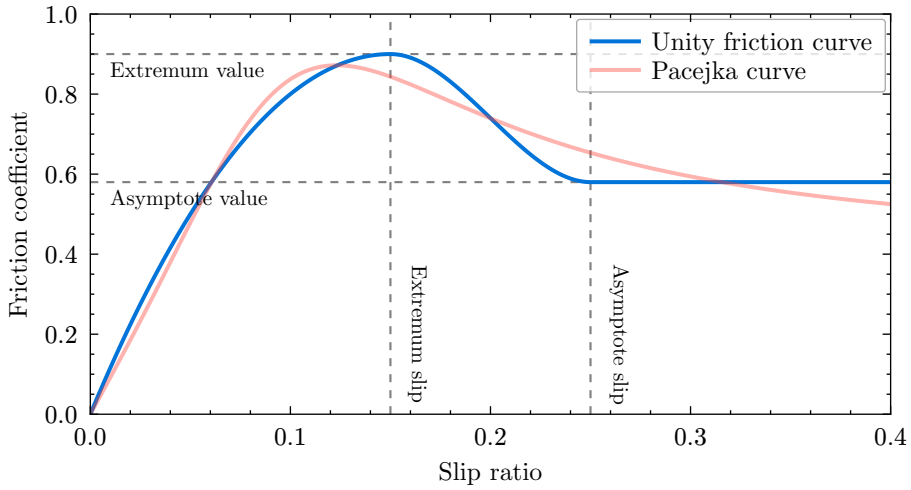


Figure 3.4: Approximated WheelCollider friction curves for the RoboRacer [41][4]

3.1.2.2. RigidBody

The car itself is modelled as a `RigidBody`, which is a Unity component that simulates the car's mass, inertia, and response to forces. The `RigidBody` ensures that the wheels

3.1.3. Track generation

During training, a variety of tracks is used to prevent the policy from overfitting to any single layout. Training on diverse environments has been shown to improve generalisation to unseen environments [44], and is closely related to domain randomisation, where simulator parameters are randomised to produce more robust policies [45], [46]

For this purpose, a track generator was implemented in C# in Unity to procedurally generate tracks. The track generator takes the following parameters as input: r - the track radius, n - the number of points to generate, $\theta_{\max}$, the maximum turn angle in a single step, ρ - the mean-reversion factor, s - the mean-reversion exponent, $\varphi_{\max}$ - the maximum heading offset, and q - the probability of flipping the driving direction. The track generator outputs a list of points that define the track.

Algorithm 3.1: Random Closed Track

procedure GENERATE-WALK(n, r, $\theta_{\max}$, ρ, s, $\varphi_{\max}$, q)
  ▷ Initialise the walk on the guide circle
  $L \leftarrow 2\pi\frac{r}{n}$
  $P_0 \leftarrow (r, 0)$
  $h \leftarrow 0$

  ▷ Random walk with a mean-reverting heading offset
  for $i = 1$ to n **do**
    $r \leftarrow \rho + (1-\rho)\left(\frac{i}{n}\right)^s$

[4]The parameter values rendered in this chart are the default values configured in the AutoDrive simulator. See Appendix A for the values.

$u \leftarrow \text{Uniform}(-\theta_{\max}, \theta_{\max})$
$h \leftarrow \text{clamp}((h+u)(1-r), -\varphi_{\max}, \varphi_{\max})$
$\alpha \leftarrow \frac{2\pi i}{n} + \frac{\pi}{2} + h$
$P_i \leftarrow P_{i-1} + L(\cos\alpha, \sin\alpha)$
end

▷ Spread the closing error evenly over the lap
$g \leftarrow P_n - P_0$
for $i = 0$ to $n-1$ **do**
$Q_i \leftarrow P_i - (\frac{i}{n})g$
end

▷ Randomly flip the driving direction, keeping Q_0 fixed
if $\text{Uniform}(0,1) < q$ **then**
$(Q_1, ..., Q_{n-1}) \leftarrow (Q_{n-1}, ..., Q_1)$
end
return Q
end

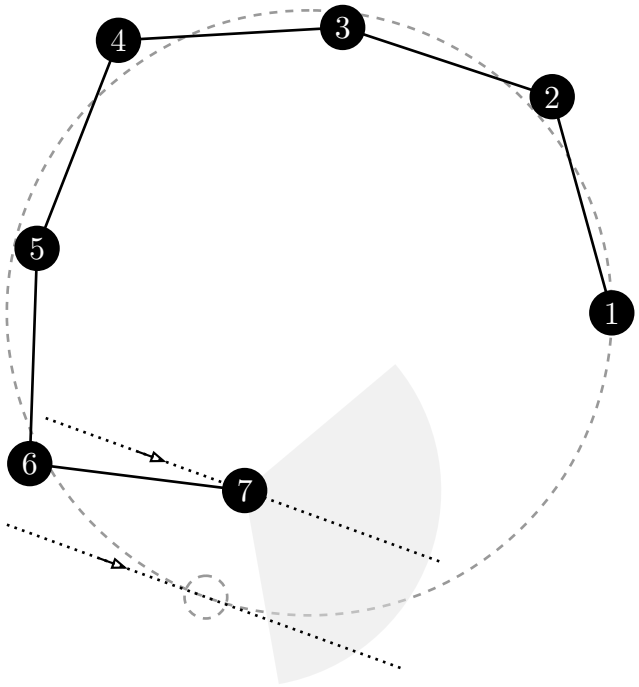


Figure 3.5: Track generation

Then, a Catmull-Rom spline is fitted to the points to create a smooth track centerline. In order to generate the inner and outer boundaries of the track, points are sampled along the centerline to generate a polygonal approximation of the track. The inner and outer boundaries are then generated by insetting/outsetting the centerline polygon by a fixed distance using the Clipper2 library [47], which extends the Vatti polygon clipping algorithm [48].

Once we have the inner and outer boundaries of the track, generating a mesh by extruding a circle or square along the splines is straightforward in Unity. An example track is shown in [ADD PLACE].

3.2. Training performance tuning

Training was performed on a single machine with an AMD Ryzen Threadripper PRO 3995WX CPU (64 cores, 128 hardware threads) and 128 GB of RAM. To ensure experiments finish in a practical time, we first measured how best to use this hardware, aiming to maximise *aggregate* throughput: environment timesteps collected and learned from per second, summed across all models training simultaneously.

We ran a grid search over the parameters in Table 3.2: concurrent trainers (2, 4, 8, 12), Python threads per trainer (8, 16) and parallel environments per trainer (4, 8, 10, 16). For each configuration, `concurrent_trainers` identical processes were launched at once, each running five PPO iterations (a 256-frame rollout plus four epochs of mini-batch updates each iteration) with checkpointing and TensorBoard logging disabled. Configurations with more than 8 trainers and 16 environments (>128 concurrent Unity instances) exhausted memory and were excluded, leaving 28 single-run configurations. Throughput per trainer was computed from the mean wall-clock time per iteration:

$$\text{throughput} = \frac{\text{rollout length} \times \text{parallel envs} \times \text{areas per env}}{\text{seconds per iteration}}$$

We report the throughput per trainer, showing how much each model slows as the machine gets busier, and summed across trainers, showing where adding trainers stops paying off.

As shown in Appendix C, total throughput peaks at 8 trainers (4172 timesteps/s with 8 environments and 16 threads) and falls at 12 (at most 3853 timesteps/s), while per-trainer throughput roughly halves from 2 to 8 trainers. The best settings use $8 \times 16 = 128$ python threads in total, matching the hardware thread count; at 12 trainers the 16-thread configurations are oversubscribed and fall behind the 8-thread ones. We therefore chose (`concurrent_trainers`, `parallel_envs`, `python_threads`) = (8, 8, 16), though its margin over the next-best configurations (4020 and 3962 timesteps/s) is within likely run-to-run variation. Using `num_areas` $\in \{8, 12\}$ per environment in this configuration did not help (3844 and 3999 timesteps/s).

By default, Unity spawns `n_cpus` garbage-collector threads; we set this to 8 with the `-gc-helper-count` launch argument for all runs.

Table 3.2: Parameters varied in the training performance grid search.

Parameter	Values considered	Description
`concurrent_trainers`	$\{2, 4, 8, 12\}$	Number of independent models training simultaneously, each in its own process.
`python_threads`	$\{8, 16\}$	Threads available to each trainer's PyTorch, OpenMP, and OpenBLAS thread pools.
`parallel_envs`	$\{4, 8, 10, 16\}$	Number of Unity environment instances per trainer. Each contains 6 training areas, so each instance steps 6 agents in parallel.

Chapter 4

Auxiliary heads

Several auxiliary heads were implemented to predict the following quantities:

Table 4.1: Auxiliary heads implemented to predict the car's forward velocity v_f, lateral velocity v_l, and angular velocity ω.

Auxiliary head(s)	Unit	Description
v_f	m/s	The car's forward velocity
v_l	m/s	The car's lateral velocity (positive is to the left)
ω	rad/s	The car's angular velocity around the verical axis (postive is counter-clockwise)
$v_{s,x}$	m/s	The car's wheel slip velocity per wheel $x \in \{\text{LF}, \text{RF}, \text{LB}, \text{RB}\}$
κ_d	$\frac{1}{m}$	Reciprocal[5] of the radius of curvature of the track at various points d along the track ahead of the car, $d \in \{0.5, 1.0, ..., 8.0\}m$

4.1. The model

The model is a feedforward neural network. Its input is a stack of the 50 most recent LIDAR scans (101 rays each), plus the previous action (steering and throttle). Since the network has no recurrent state, the stacked scans are the only source of temporal information.

The scans are processed by four 1D convolutional layers with ReLU activations. The convolutions slide along the ray axis and treat the 50 time steps as input channels, so the first layer combines the whole history at each angular position and turns it into a set of learned feature channels. The remaining three layers work on these features and only add spatial context along the ray axis. The output is flattened, concatenated with

[5]The reciprocal of the radius of curvature is used to prevent undefined values when the track is straight.

the previous action, and passed through three fully connected layers (128, 72 and 64 units) with ReLU activations and a final layer normalization.

The 64-dimensional output is shared by all heads. The last layer ("out" in Figure 4.1) is a linear layer that produces the outputs of every head at once: 4 for the policy (mean and scale for steering and throttle), 1 for the value estimate, and 27 for the auxiliary prediction targets, which are only used as training signals.

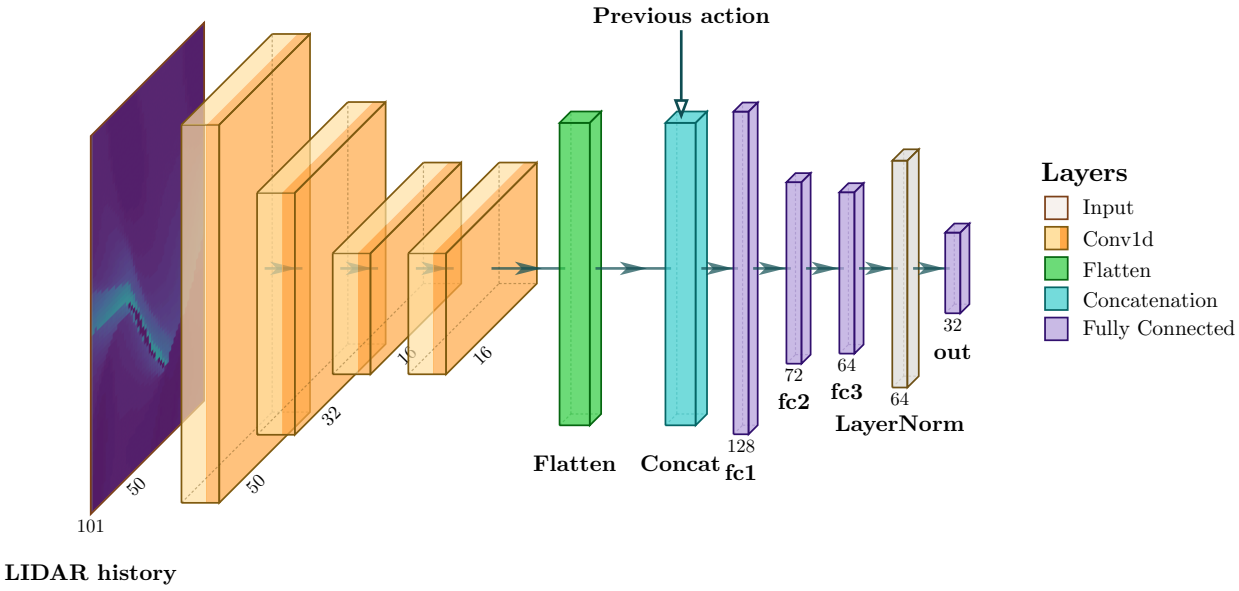


Figure 4.1: Neural network architecture

Chapter 5

Experiments & results

5.1. Probing intermediate layers for auxiliary information

The intuition rests on the geometry of the feature space. Each input is mapped to a point $\boldsymbol{z}_i$ in the space of `fc3` activations, and a linear probe predicts $\hat{\boldsymbol{y}}_i = \boldsymbol{W}\boldsymbol{z}_i + \boldsymbol{b}$. Each output of the probe is therefore a weighted sum of the features, which is equivalent to projecting $\boldsymbol{z}_i$ onto a single direction in the feature space. If the model has learnt to encode a quantity along some direction, so that moving along that direction corresponds to a change in the quantity, the probe can learn that direction and read the quantity off directly. In this case the information is *linearly decodable*, and the probe will predict $\boldsymbol{y}_i$ well. If instead the information is absent from the features, no direction exists that correlates with $\boldsymbol{y}_i$, and the probe can do no better than a trivial baseline such as predicting the mean of $\boldsymbol{y}$ (an R^2 near zero).

Because the probe is restricted to linear projections, it cannot construct new features of its own. Any predictive performance must therefore come from structure already present in the representation, rather than from the probe doing the learning. This makes probe accuracy a measure of how explicitly, not merely whether, the information is represented at that layer.

To construct the probing dataset, we perform a rollout using baseline model for 2048 steps (82 s of simulated time) in each of 32 parallel agents. At every step we record each agent's LIDAR observation $\boldsymbol{o}_{i,t} \in \mathbb{R}^{101}$ together with the corresponding labelled information $\boldsymbol{y}_{i,t}$, giving an observation tensor $\boldsymbol{O} \in \mathbb{R}^{32\times 2048\times 101}$. Then, for each checkpoint of each training run of the baseline model, each observation is passed through the trunk to obtain the `fc3` activations $\boldsymbol{z}_{i,t}$, which serve as inputs to the linear probe. We fit the probe on the rollouts of 25 agents (≈78% train split) and evaluate it on the remaining 7, so that the train and test sets share no trajectories or track layouts.

Interestingly, we observe that even before the first epoch of gradient descent (i.e. with LeCun-randomly initialised weights), some features are already encoded by the model, as shown in Figure 5.1. This is especially true for the car's lateral velocity v_l and angular velocity ω_y, which is predicted with a mean out-of-sample R^2 of $\overline{R}^2 = 92\%$ and $\overline{R}^2 = 85\%$ respectively. [We hypothesise that this is because the car's lateral and angular velocities are strongly correlated with the LIDAR observations, and any LeCun-random projection of these observations will maintain some of this correlation. Then, with many such observations (101 rays x 50 time steps), this correlation is strongly preserved.]. A similar effect has been observed in [30], where the authors show that, on a small MNIST convolutional neural network, the the probe error decreases with the number of layers, even on randomly initialised weights.

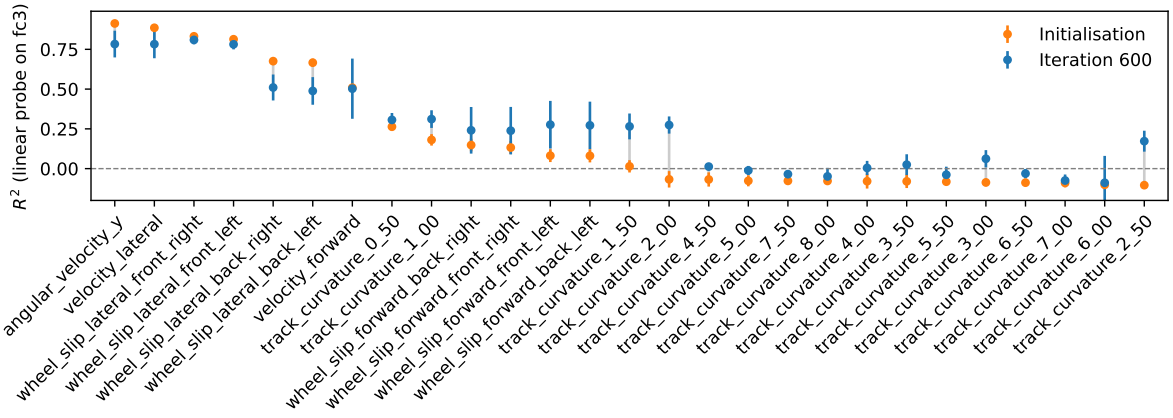
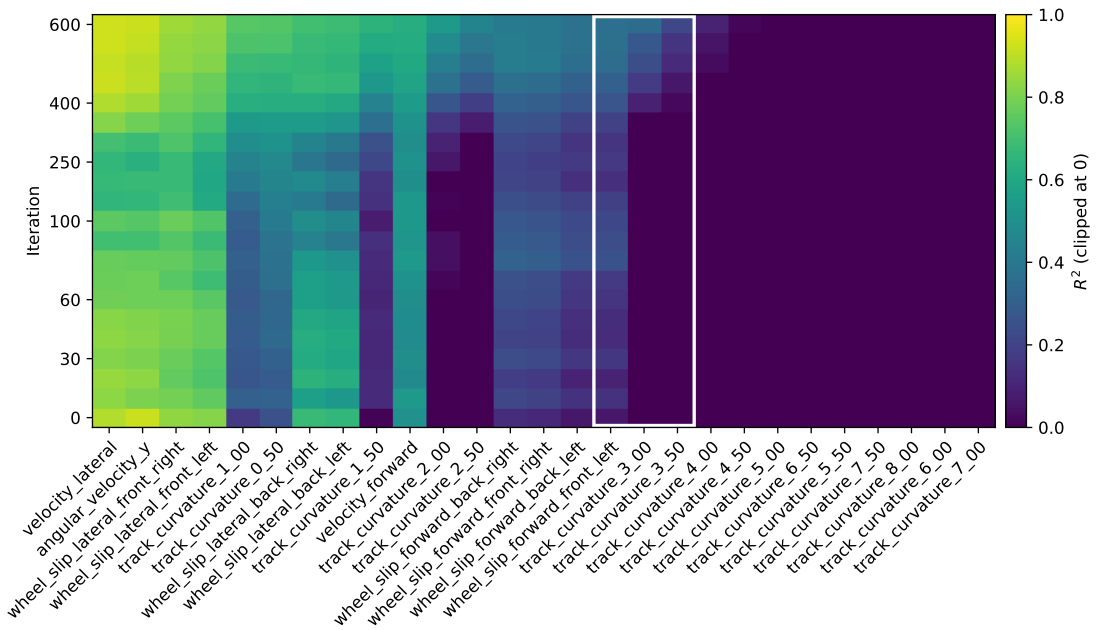


Figure 5.1: Out-of-sample R^2 scores of linear regression probing of the final layer of the trunk (`fc3`) on the baseline model



5.2. Meeting questions

First, explain the findings I have. Then:

1. Should I be reporting p-values/statistical significance for the probe deltas?
2. Should I be doing a False Discovery Rate (FDR) correction for testing multiple probes?
3. How should I present this information - we have 4 axes: (target, aux heads active, aux heads weight, iteration) with a scalar in each point? Maybe, a grid (target x aux heads active) of line plots (iteration on x axis, R^2 with variance on Y axis)?
4. Going forward, should I test H2H or rather add uncertainty weighting?

- Clarify naming of “auxiliary features” => “auxiliary targets”

5.3. Learned features after auxiliary learning

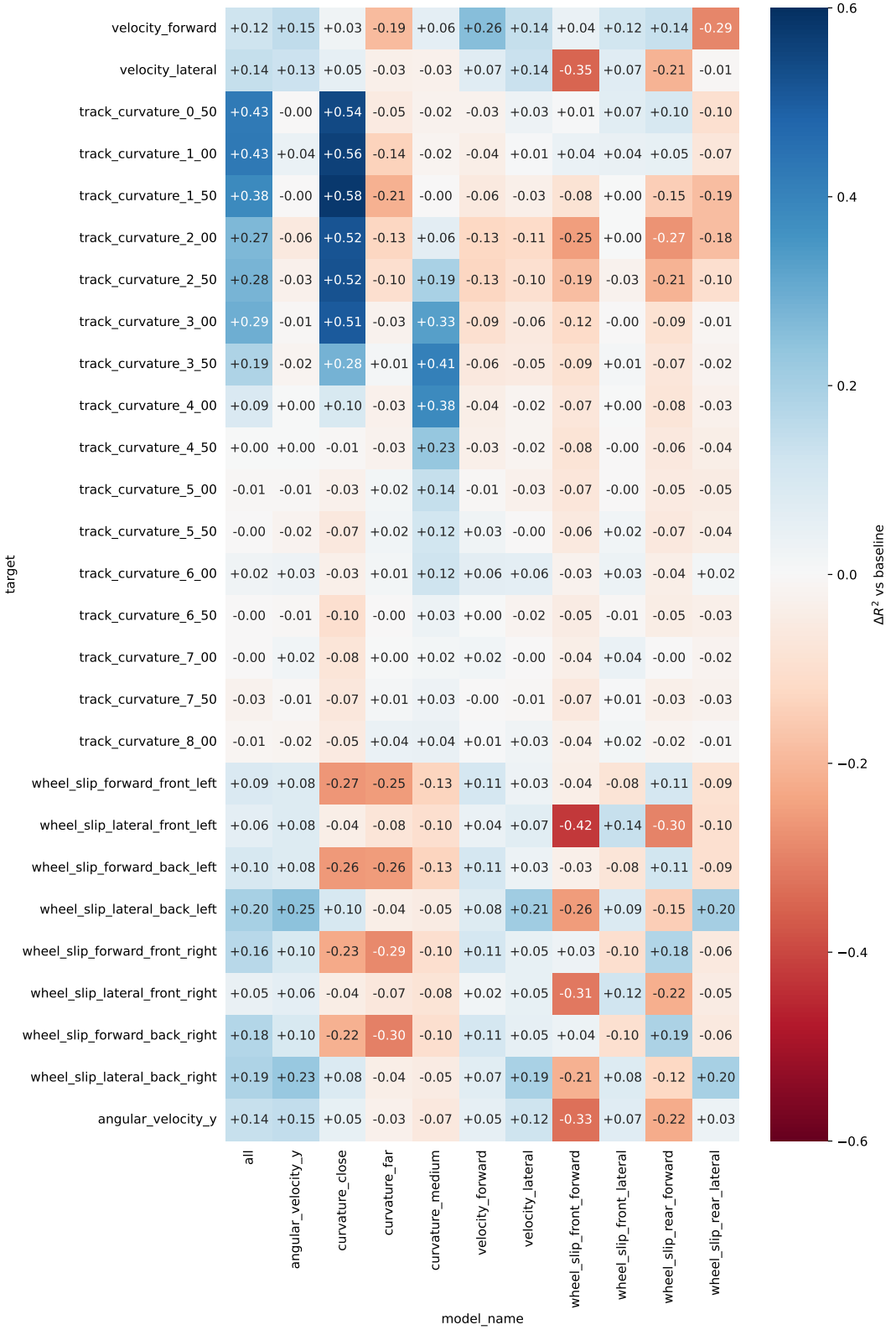


Figure 5.3: Mean out-of-sample R^2 deltas compared to the baseline model, showing how much more (or less) linearly decodable each quantity is after auxiliary learning

5.4. Primary Task Performance

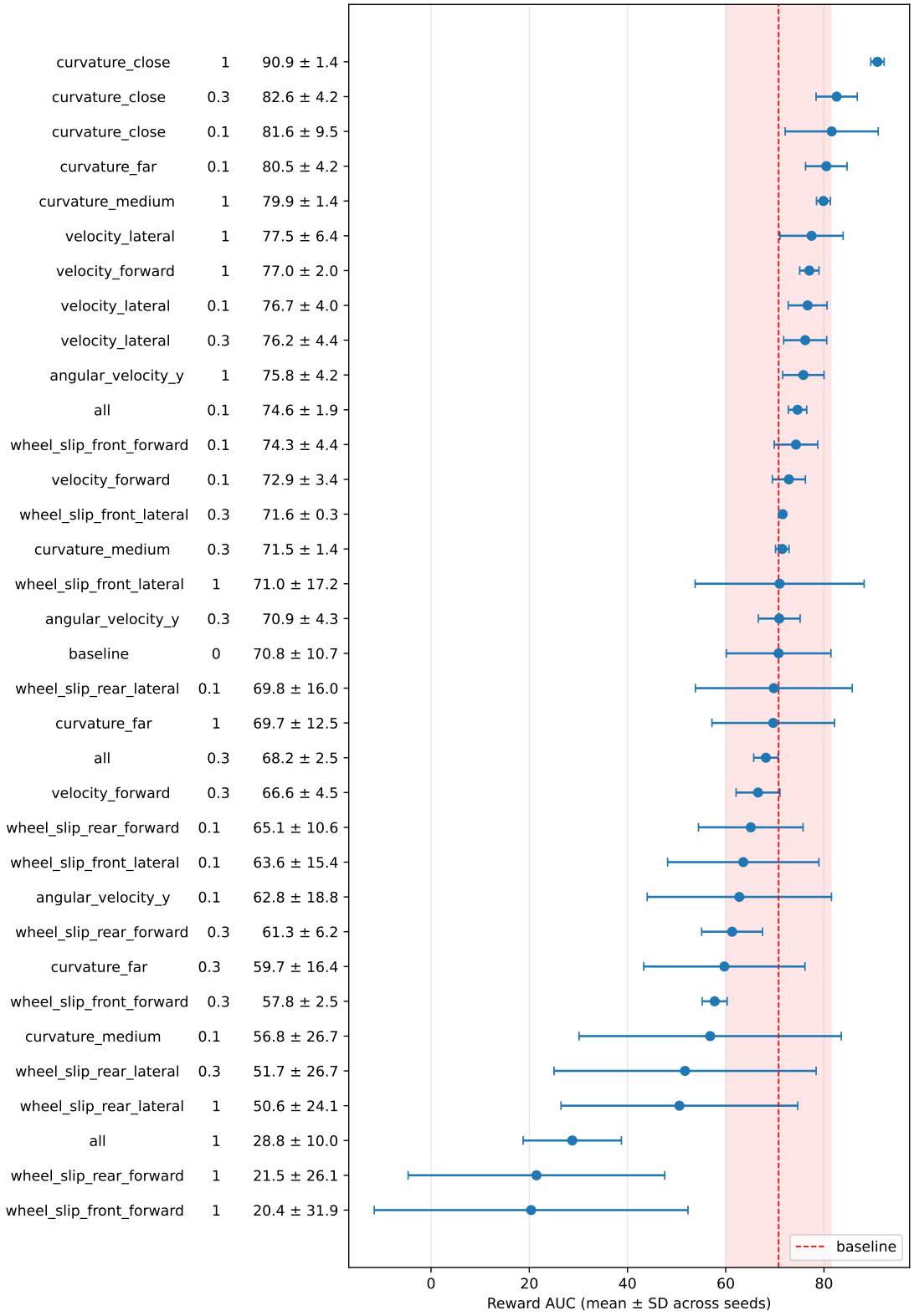


Figure 5.4: AUC for the primary task reward r_t by auxiliary head

5.4.1. Variation in Primary Task Performance

5.5. Normalisation of ego-agent privledged information

Prior to applying auxiliary learning, the privileged information must be normalised. The auxiliary loss is a sum over the individual prediction targets, so a target with a larger numerical range contributes larger errors and gradients, and can dominate the shared representation at the expense of the others. Training with unnormalised targets makes hyperparameter tuning [49], and training more difficult [50].

Kendall et al. proposes a solution to this problem by learning a log-variance $\log \sigma_i$ for each loss L_i [27]. Because it is expensive to perform hyperparameter searches across the loss weights $\{w_1, ..., w_N\}$,.

Epistemic uncertainty is the uncertainty that comes as a result of lack of training data.

Aleatoric uncertainty is that which cannot be reduced

An alternative method is [51], which dynamically scales the weights of each loss term to balance the $\|\delta_W L_i\|_2$

Chapter 6

What I did

- **Car always going full throttle + full steering**: This is because tanhnormal saturates at 1, so the model cannot learn when the output is close to 1 or -1. This is a problem because the car is always going full throttle, and the steering angle is often close to 1 or -1. The model cannot learn to output values further away. So, the scale of weights & biases was changed to start in the center
- **LIDAR angle pointing up, causing large lidar values input** - increase wall height, mask "miss" rays to 0
- **Huber loss is invariant** to scale if the parameter beta is updated:

$$L_\beta\left(\frac{x}{\alpha}, \frac{\hat{x}}{\alpha}\right) = \frac{L_{\alpha*\beta}(x, \hat{x})}{\alpha^2}$$

- **Regression based on the hidden layer output of the PPO model**: The PPO model is trained to predict the value function and policy, so the hidden layer output contains information about the state of the environment. This information can be used to predict the auxiliary targets. The regression results show that the hidden layer output contains information about the auxiliary targets, and that this information can be extracted by a simple linear regression model. [30]
- By default, BLAS spawns 128 threads. Did hyperparameter tuning to find the best combination of (num_pytorch_threads, num_parallel_trainers, num_envs_per_trainer). Also Unity spawns 128 GC threads, which can be controlled via `"-job-worker-count", str(num_unity_threads), "-gc-helper-count", str(num_unity_threads)`

Chapter 7

Conclusions and recommendations

Bibliography

[1] B. Olofsson and L. Nielsen, "Using Crash Databases to Predict Effectiveness of New Autonomous Vehicle Maneuvers for Lane-Departure Injury Reduction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 6, pp. 3479–3490, 2021, doi: 10.1109/TITS.2020.2983553.

[2] C. J. G. V. Driel and B. V. Arem, "The Impact of a Congestion Assistant on Traffic Flow Efficiency and Safety in Congested Traffic Caused by a Lane Drop," *Journal of Intelligent Transportation Systems*, vol. 14, no. 4, pp. 197–208, 2010, doi: 10.1080/15472450.2010.516226.

[3] H. Igliński and M. Babiak, "Analysis of the Potential of Autonomous Vehicles in Reducing the Emissions of Greenhouse Gases in Road Transport," *Procedia Engineering*, vol. 192, pp. 353–358, 2017, doi: https://doi.org/10.1016/j.proeng.2017.06.061.

[4] J. Chen, S. E. Li, and M. Tomizuka, "Interpretable End-to-end Urban Autonomous Driving with Latent Deep Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2001.08726

[5] B. Rafiee, J. Jin, J. Luo, and A. White, "What makes useful auxiliary tasks in reinforcement learning: investigating the effect of the target policy." [Online]. Available: https://arxiv.org/abs/2204.00565

[6] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.

[7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[8] R. Bellman, "A Markovian Decision Process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.

[9] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," Sep. 2015. [Online]. Available: https://ar5iv.arxiv.org/html/1509.02971

[10] S. P. Singh, T. Jaakkola, and M. I. Jordan, "Learning Without State-Estimation in Partially Observable Markovian Decision Processes," in *Machine Learning*

Proceedings 1994, W. W. Cohen and H. Hirsh, Eds., San Francisco (CA): Morgan Kaufmann, 1994, pp. 284–292. doi: https://doi.org/10.1016/B978-1-55860-335-6.50042-8.

[11] P. Thomas, "Bias in Natural Actor-Critic Algorithms," in *Proceedings of the 31st International Conference on Machine Learning*, E. P. Xing and T. Jebara, Eds., in Proceedings of Machine Learning Research, vol. 32. Beijing, China: PMLR, 2014, pp. 441–448. [Online]. Available: https://proceedings.mlr.press/v32/thomas14.html

[12] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, May 1992, doi: 10.1007/BF00992696.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: https://arxiv.org/abs/1707.06347

[14] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation." [Online]. Available: https://arxiv.org/abs/1506.02438

[15] M. Jaderberg *et al.*, "Reinforcement Learning with Unsupervised Auxiliary Tasks," in *International Conference on Learning Representations (ICLR 2017)*, 2017. doi: 10.48550/arXiv.1611.05397.

[16] E. Hüllermeier, T. Fober, and M. Mernberger, "Inductive Bias," in *Encyclopedia of Systems Biology*, W. Dubitzky, O. Wolkenhauer, K.-H. Cho, and H. Yokota, Eds., New York, NY: Springer, 2013, p. 1018. doi: 10.1007/978-1-4419-9863-7_927.

[17] R. Caruana, "Multitask Learning," *Machine Learning*, vol. 28, pp. 41–75, Jul. 1997, doi: 10.1023/A:1007379606734.

[18] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, in ICML'17. Sydney, NSW, Australia: JMLR.org, 2017, pp. 2778–2787.

[19] A. Y. Ng, D. Harada, and S. J. Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning*, in ICML '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, pp. 278–287.

[20] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," *arXiv preprint*, Jul. 2019, doi: 10.48550/arXiv.1907.10902.

[21] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[22] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," in *Advances in Neural Information Processing Systems*, 2012, pp. 2951–2959. [Online]. Available: https://arxiv.org/abs/1206.2944

[23] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, 2011. [Online]. Available: https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html

[24] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research*, vol. 18, pp. 1–52, 2018, [Online]. Available: https://arxiv.org/abs/1603.06560

[25] L. Li *et al.*, "A System for Massively Parallel Hyperparameter Tuning," in *Proceedings of Machine Learning and Systems*, 2020, pp. 230–246. [Online]. Available: https://arxiv.org/abs/1810.05934

[26] A. Kendall and Y. Gal, "What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?," *arXiv preprint*, Mar. 2017, [Online]. Available: https://arxiv.org/abs/1703.04977

[27] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2018, pp. 7482–7491. [Online]. Available: https://arxiv.org/abs/1705.07115

[28] L. Kirchdorfer, T. Sesterhenn, C. Bartelt, H. Stuckenschmidt, L. Schott, and J. M. Köhler, "Investigating Uncertainty Weighting for Multi-Task Learning: Insights and Analytical Alternative," *International Journal of Computer Vision*, vol. 134, no. 8, Dec. 2025, doi: 10.1007/s11263-025-02625-x.

[29] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, Mar. 1964, doi: 10.1214/aoms/1177703732.

[30] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes." [Online]. Available: https://arxiv.org/abs/1610.01644

[31] A. Anand, E. Racah, S. Ozair, Y. Bengio, M.-A. Côté, and R. D. Hjelm, "Unsupervised State Representation Learning in Atari." [Online]. Available: https://arxiv.org/abs/1906.08226

[32] Y. Belinkov, "Probing Classifiers: Promises, Shortcomings, and Advances." [Online]. Available: https://arxiv.org/abs/2102.12452

[33] T. McGrath *et al.*, "Acquisition of chess knowledge in AlphaZero," *Proceedings of the National Academy of Sciences*, vol. 119, no. 47, p. e2206625119, 2022, doi: 10.1073/pnas.2206625119.

[34] W. Gurnee and M. Tegmark, "Language Models Represent Space and Time." [Online]. Available: https://arxiv.org/abs/2310.02207

[35] G. Lample and D. S. Chaplot, "Playing FPS Games with Deep Reinforcement Learning," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, AAAI Press, 2017, pp. 2140–2146. [Online]. Available: https://arxiv.org/abs/1609.05521

[36] Unity Technologies, *Unity ML-Agents Toolkit*. (2017). GitHub. Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/Unity-Technologies/ml-agents

[37] A. Juliani *et al.*, "Unity: A General Platform for Intelligent Agents," 2018. [Online]. Available: https://arxiv.org/abs/1809.02627

[38] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning," in *NeurIPS 2019 Competition and Demonstration Track*, 2020, pp. 77–89.

[39] T. V. Samak, C. V. Samak, and M. Xie, "AutoDRIVE Simulator: A Simulator for Scaled Autonomous Vehicle Research and Education," in *2021 2nd International Conference on Control, Robotics and Intelligent System*, in CCRIS'21. Qingdao, China: Association for Computing Machinery, 2021, pp. 1–5. doi: 10.1145/3483845.3483846.

[40] "Customised Usage Example." Accessed: Oct. 14, 2026. [Online]. Available: https://f1tenth-gym.readthedocs.io/en/v1.0.0/customized_usage.html#random-track-generator-beta

[41] "Unity Documentation." [Online]. Available: https://docs.unity3d.com/6000.5/Documentation/ScriptReference/WheelFrictionCurve.html

[42] H. B. Pacejka and E. Bakker, "THE MAGIC FORMULA TYRE MODEL," *Vehicle System Dynamics*, vol. 21, no. sup001, pp. 1–18, 1992, doi: 10.1080/00423119208969994.

[43] H. Pacejka, *Tyre and Vehicle Dynamics*. in Automotive Engineering Series. Elsevier Science & Technology Books, 2002. [Online]. Available: https://books.google.co.za/books?id=pQNUAAAAMAAJ

[44] K. Cobbe, O. Klimov, C. Hesse, T. Kim, and J. Schulman, "Quantifying Generalization in Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2019, pp. 1282–1289.

[45] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[46] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[47] A. Johnson, *Clipper2: Polygon Clipping, Offsetting & Triangulation in C++, C# and Delphi*. (2022). GitHub. Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/AngusJohnson/Clipper2

[48] B. R. Vatti, "A generic solution to polygon clipping," *Communications of the ACM*, vol. 35, no. 7, pp. 56–63, Jul. 1992, [Online]. Available: https://dl.acm.org/doi/10.1145/129902.129906

[49] H. van Hasselt, A. Guez, M. Hessel, V. Mnih, and D. Silver, "Learning values across many orders of magnitude," in *Advances in Neural Information Processing Systems*, 2016.

[50] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998, doi: 10.1109/5.726791.

[51] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks," Nov. 2017. [Online]. Available: https://arxiv.org/abs/1711.02257

Appendix

Appendix A WheelCollider properties

Table 7.1: Relevant parameters of the WheelCollider model. * = forward & sideways

Property	Description	Value for RoboRacer
`radius`	Radius of the wheel	5.8 cm
`width`	Radius of the wheel	5.8 cm
`mass`	Mass of each wheel; determines its rotational inertia	109 g
`suspension_distance`	Maximum travel of the suspension along the wheel's local Y axis	5 cm
`suspension_spring`	Spring stiffness pulling the wheel toward its target position	500 N/m
`suspension_damper`	Damping that resists the speed of suspension movement	100 (N · s/m)
`extremum_slip`*	Slip at which the friction curve peaks (maximum grip)	0.15 fwd / 0.01 side
`extremum_value`*	Friction at the peak of the curve	0.9 fwd / 1 side
`asymptote_slip`*	Slip beyond which the tyre is fully sliding	0.25 fwd / 0.1 side
`asymptote_value`*	Friction while sliding	0.58 fwd / 0.5 side
`stiffness`*	Multiplier applied to the whole friction curve	0.8 fwd / 1 side

Appendix B Auxiliary Head Regression Results

Appendix C Training Performance Benchmark Results

Table 7.2: Training performance results, sorted by throughput.

Concurrent trainers	Python threads	Parallel envs[6]	Throughput (timesteps/sec)	Speedup vs realtime[7]
8	16	8	4172.37	166.89x
8	16	4	4020.17	160.81x
8	16	8	3999.46	159.98x
8	16	10	3962.12	158.48x
12	8	4	3853.04	154.12x
8	16	8	3843.59	153.74x
12	16	10	3805.19	152.21x
8	8	4	3802.34	152.09x
12	8	8	3751.41	150.06x
12	8	10	3717.67	148.71x
12	16	8	3582.39	143.3x
4	16	10	3428.19	137.13x
12	16	4	3302.99	132.12x
8	8	8	3207.98	128.32x
4	16	8	3201.63	128.07x
8	8	10	3164.72	126.59x
4	16	4	2684.4	107.38x
4	8	4	2414.64	96.59x
2	16	10	2206.84	88.27x
4	8	16	2086.99	83.48x
4	16	16	2075.62	83.02x
2	16	8	2034.77	81.39x
4	8	10	1884.89	75.4x
4	8	8	1830.29	73.21x
2	16	4	1557.77	62.31x
2	8	4	1385.09	55.4x
2	8	16	1122.47	44.9x
2	16	16	1122.1	44.88x
2	8	10	1013.7	40.55x
2	8	8	977.93	39.12x

[6]It was not possible to run 16 parallel envs on 12 trainers, because the machine ran out of memory.
[7]Assuming 25 Hz decision frequency